

Tarea 4

Tarea 4



Embarcaciones

Programación Orientada a Objetos

CE Desarrollo de aplicaciones en lenguaje Python



UT2: Tarea 4
Embarcaciones



1.- Descripción de la tarea



Caso práctico

La empresa que gestionaba la escuela de vela de la tarea de la unidad anterior ha ampliado su capacidad y ahora gestiona además un puerto deportivo, por lo que ha encargado a **Trassierra Programación** la implantación de algún mecanismo que permita llevar un control no solo de los barcos veleros y las regatas como hasta ahora, sino que añade la gestión de lanchas motoras.

Para ello es preciso crear una clase padre para prepararse ante futuras ampliaciones, que la clase velero sea una subclase de esta y que además se cree una nueva subclase para gestionar las lanchas y que guarde información similar a la que se guardaba con los veleros.

Para modela la clase **Velero**:

- La mayoría de los atributos que anteriormente pertenecían a los veleros, serían los atributos de una superclase llamada "**Embarcacion**", salvo los atributos propios de un velero, que son los **mástiles**.
 - Hay que tener en cuenta que una lancha tiene **motores**, pero estas consumen **combustible**.
 - Como hay varios métodos que se encargan de la navegación, se debería crear una interfaz "**Navegable**" para establecer estos métodos.
 - Además, habría que identificar las embarcaciones capaces de competir en una regata y crear una interfaz "**Regateable**" para establecer estos métodos.
-



Pietrofotografie

<<https://pixabay.com/es/photos/puerto-barcos-atardecer-hora-dorada-6496782//>> (Pixabay License
<<https://pixabay.com/es/service/license/>>)



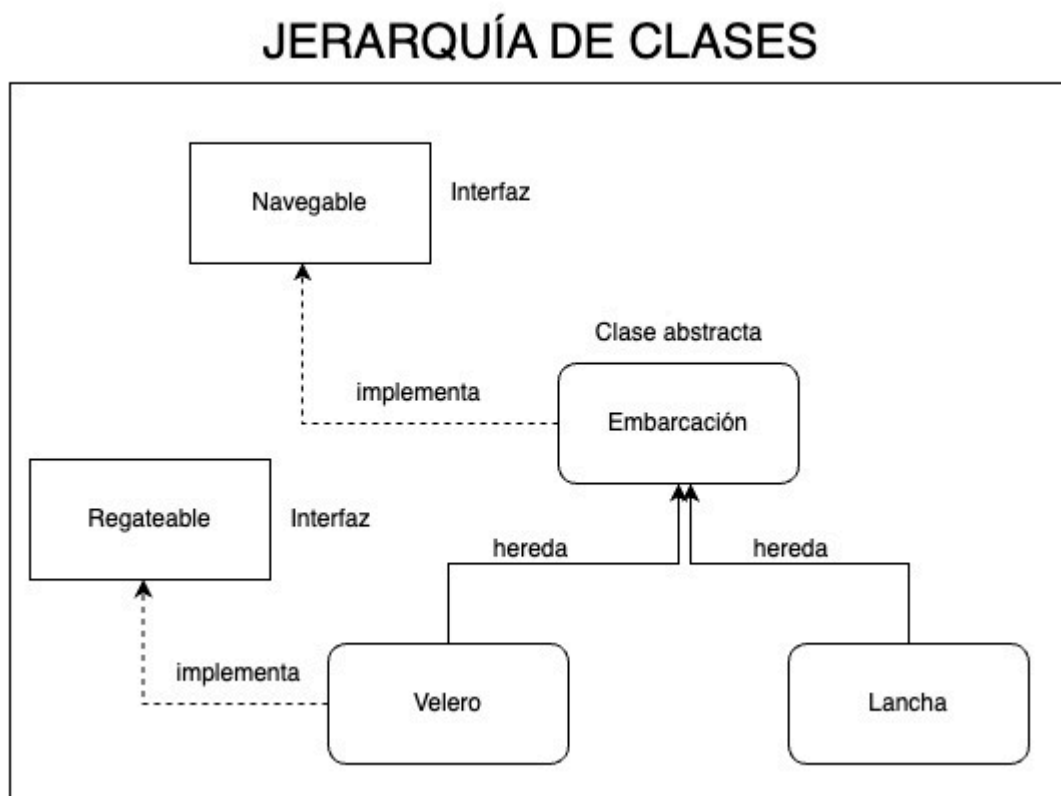
¿Qué te pedimos que hagas?

¿Qué te pedimos que hagas?

La tarea consiste en la implementación de una **jerarquía de clases basada en los tipos de embarcaciones** que hay en un **puerto deportivo**. Nos basaremos en la clase abstracta *Embarcacion* y, a partir de ella, se implementarán dos serie de clases derivadas (*Velero* y *Lancha*) con unas características (atributos) y comportamientos (métodos) específicos. De esta manera, practicaremos el uso de una gran parte de los mecanismos y recursos relacionados con el **desarrollo avanzado de clases** estudiados en esta unidad (herencia y composición de clases, redefinición y ampliación de métodos, implementación de interfaces, polimorfismo, ligadura dinámica, etc.).

Algunas de las clases que tendrás que implementar serán **abstractas** (no se podrán instanciar objetos a partir de ellas, tan solo implementar otras subclases derivadas a partir de ellas) mientras que otras sí serán **instanciables**. Por otro lado, se te pedirá que definas dos **interfaces** para especificar, en una de ellas, los métodos que se encarguen de la navegación, y en la otra, los encargados de la regata.

La jerarquía de clases debe quedar tal que así:



Elaboración propia (CC BY-NC-SA <<http://creativecommons.org/licenses/?lang=es>>)

A lo largo de los siguientes apartados de la tarea se irán explicando detalladamente

cada uno de los elementos (clases e interfaces) que tendrás que implementar, proporcionándote toda la información que necesitas para que puedas desarrollarlos sin dificultad.

1.1.- Clase Embarcacion



Clase Embarcacion

En esta clase definiremos los atributos mínimos para representar las características elementales de cualquier embarcación dentro de nuestro puerto deportivo.

Tenemos que tener en cuenta que de esta clase va a ser nuestra **superclase**, por lo que el resto de clases que implementemos deben heredar lo máximo posible de esta con la finalidad de reutilizar la mayor cantidad de código. Además, **no debe ser instanciable**, ya que no tiene sentido crear un objeto de este tipo.

La clase Embarcacion implementa la interfaz Navegable y deberá tener los **atributos, constructores y métodos** que describiremos en los siguientes puntos.



Angeleses

<https://pixabay.com/es/photos/puerto-mar%C3%ADtimo-puerto-embarcaciones-3786662/> (Pixabay License
<https://pixabay.com/es/service/license/>)

1.1.1.- Atributos



Atributos

En la clase `Embarcacion` se definen los atributos mínimos para representar las características elementales de cualquier **barco** que esté en el **puerto deportivo**.

*Todos los atributos deben ser **accesibles únicamente por la clase Embarcacion y por sus clases hijas**, exceptuando las **constantes**, que deben ser públicas tal y como se indica más abajo.*

Algunos atributos tendrán un valor que se definirá al instanciarse el objeto y ya nunca modificarán su contenido durante toda la vida del objeto (constantes). Podría decirse que definen algo así como la "naturaleza inmutable" o **características intrínsecas del objeto**. **Sólo se podrá acceder a estos atributos desde la clase Embarcacion:**

- **nombre** del barco.
- **número máximo de tripulantes** que admite el barco.

Otros atributos sí irán cambiando a lo largo de la vida del objeto y de alguna manera representarán el **estado del objeto** en cada instante. Por un lado, estará el siguiente atributo referente al estado del barco. **Se podrá acceder a estos atributos desde cualquier clase que herede de la clase Embarcacion:**

- si **está navegando** o no.

Y por otro, los atributos relacionados con la navegación:

- la **velocidad** de navegación (en nudos).
- el **nombre del patrón** del barco durante la navegación.
- el **rumbo** del barco durante la navegación.
- el **número de tripulantes** del barco durante la navegación (exceptuando el patrón).
- el **tiempo total de navegación** del barco.

Además, también se dispondrá de otros atributos que pertenecerán a la clase (atributos de clase), más que a un objeto en particular, compartiendo su valor para todos los objetos. Estos atributos tendrán sentido siempre, independientemente de que existan o no instancias de la clase (**atributos de clase**) y se accederá a ellos a través del nombre de la clase (y no de una referencia a un objeto). **Sólo se podrá acceder a estos atributos desde la clase Embarcacion:**

1. el **número de barcos** creados hasta ese momento.
2. el **número de barcos que están navegando**.

3. el **tiempo total de navegación** acumulado por todos los barcos.

Por último, junto con esos atributos de clase variables, la clase también va a contener una serie de **constantes** que serán de utilidad a la hora de realizar comprobaciones o asignaciones por omisión. Estas constantes serán **públicas** y tendrá acceso a ellas cualquier programador que utilice la clase para así poder asegurarse en sus programas de que no se superan los umbrales permitidos:

1. **nombre del patrón** por defecto durante la navegación (Sin patrón).
Atributo **PATRON_POR_DEFECTO**;
2. **rumbo** por defecto durante la navegación (Sin rumbo).
Atributo **RUMBO_POR_DEFECTO**;
3. **número mínimo de tripulantes** durante la navegación, excluyendo el patrón (0).
Atributo **MIN_TRIPULANTES**;

Se te proporciona explícitamente el **nombre de estos últimos atributos** porque van a ser **públicos** y ya hay programas desarrollados por otras personas que están usando esos valores (son valores de "interfaz" y, por tanto, públicos). En consecuencia, hemos tenido que ponernos de acuerdo con el resto de programadores del equipo de desarrollo en los nombres de todos aquellos elementos que sean públicos (nombre de la clase, nombre de los atributos públicos, nombres de los métodos públicos, orden y tipo de los parámetros en los métodos públicos, etc.). Para el resto de elementos de la clase (miembros privados, variables locales, etc.) se podrán utilizar los identificadores que consideres más oportunos, pues estarán ocultos a otros programadores y solo serán accesibles desde el código de los métodos de la propia clase.

1.1.2.- Constructores



Constructores

Hay que implementar un **constructor con dos parámetros: nombre y número máximo de tripulantes**.

Este constructor creará un nuevo objeto instancia de la clase con la información que se le proporciona a través de sus parámetros, con un nombre que cumplirá las reglas indicadas en el apartado anterior.

Si alguno de los parámetros del constructor es incorrecto o se produce alguna circunstancia que no permita que se instancie el objeto, deberá lanzarse una **excepción** y se detendrá el proceso de instanciación del objeto. En este caso, las posibles excepciones que podrían lanzarse son:

- si el constructor no recibe un nombre, el mensaje de error de la excepción debería ser del tipo **"El nombre de la embarcación es obligatorio."**
- si el constructor recibe una cadena vacía en nombre, el mensaje de error de la excepción debería ser del tipo **"El nombre de la embarcación no puede estar vacío."**
- si el número de tripulantes inferior al mínimo permitido, el mensaje de error de la excepción debería ser del tipo **"El número de tripulantes debe ser, como mínimo, xxx."**, donde xxx representaría el número mínimo de tripulantes permitido.

Si los parámetros son correctos, se procederá a asignar los **valores iniciales de los atributos de objeto** así como a **actualizar los atributos de clase** que sea necesario modificar. En el caso de los **atributos relacionados con la navegación**, debes tener en cuenta que cuando se instancia una embarcación se debe crear sin estar navegando, por tanto deberás inicializar estos atributos con los **valores por defecto** o **valores mínimos**.

1.1.3.- Métodos



Métodos

1. *Añade a la clase Embarcacion los siguientes **métodos de consulta (getters)** para poder obtener la siguiente información de objeto:*

1. `get_nombre_barco()`, que devuelva el **nombre** de la embarcación;
2. `get_num_max_tripulantes()`, que devuelva el **número máximo de tripulantes** que admite la embarcación;
3. `is_navegando()`, que indique si la embarcación **está navegando o no**;
4. `get_velocidad()`, que devuelva la **velocidad** (en nudos) durante la navegación;
5. `get_rumbo()`, que devuelva el **rumbo** durante la navegación;
6. `get_patron()`, que devuelva el **nombre del patrón** durante la navegación;
7. `get_tripulacion()`, que devuelva el **número de tripulantes** durante la navegación;
8. `get_tiempo_total_navegacion()`, que devuelva el **tiempo total de navegación del barco** en minutos;

2. *Además de esos métodos consultores sobre el estado del objeto, también habrá que implementar algunos **métodos que devuelvan información genérica o global sobre la clase (de clase)**:*

1. `get_num_barcos()`, que devuelva la **cantidad de embarcaciones creadas** hasta el momento.
2. `get_num_barcos_navegando()`, que devuelva la **cantidad de embarcaciones navegando** en ese momento.
3. `get_tiempo_total_navegacion_acumulado()`, que devuelva el **tiempo total de minutos de navegación acumulados por todas las embarcaciones**.

3. *También tendremos que implementar un **método para modificar atributos de nuestra clase**:*

Para representar el cambio de rumbo de una embarcación que está navegando implementaremos el siguiente método:

- `set_rumbo()`, que servirá para representar el cambio de rumbo de una embarcación que está navegando. Este método recibirá por parámetro el nuevo rumbo y no devolverá nada.

Debes tener en cuenta que **si la embarcación no está navegando no se debería poder cambiar el rumbo**, y habría que lanzar una **excepción** con el mensaje de error "**La embarcación xxx no está navegando, no se puede cambiar el rumbo.**", donde xxx será el nombre de la embarcación.

Además, si **el parámetro rumbo es incorrecto**, también deberá **lanzarse una excepción**. En este caso, las posibles excepciones que podrían lanzarse son:

- si se quiere cambiar al mismo rumbo en el que está navegando la embarcación actualmente, el mensaje de error de la excepción debería ser del tipo **"La embarcación xxx ya está navegando con ese rumbo (zzz), debes indicar un rumbo distinto para poder modificarlo."**, donde xxx será el nombre de la embarcación y zzz su rumbo actual.

Si se ha podido realizar el cambio de rumbo, el método tendrá que **actualizar apropiadamente los atributos de objeto y/o de clase necesarios** para representar que se ha efectuado el cambio de rumbo.

*4. Además implementaremos los **métodos de acción** relacionados con la interfaz "Navegable":*

Para poder **iniciar la navegación** y posteriormente **pararla** habrá que implementar los siguientes métodos que se encargarán de actualizar apropiadamente todos los atributos de estado necesarios, tanto de objeto como de clase, para que se realice adecuadamente el proceso de navegación:

iniciar_navegacion(), que servirá para **iniciar la navegación de una embarcación**. Este método recibirá por parámetro la velocidad, el rumbo, el patrón y el número de tripulantes y no devolverá nada. Posteriormente será redefinido al implementar en la clase la interfaz "INavegable".

Como siempre, tendremos que tener en cuenta si alguno de los **parámetros del método es incorrecto** para **lanzar las correspondientes excepciones**:

- si se intenta iniciar la navegación de una embarcación que ya está navegando, se debería lanzar una excepción con el mensaje **"La embarcación xxx ya está navegando y se encuentra fuera de puerto."**, donde xxx será el nombre de la embarcación.
- si el método recibe una cadena vacía en rumbo, el mensaje de error de la excepción debería ser del tipo **"Debes indicar el rumbo para iniciar la navegación."**
- si el método recibe una cadena vacía en el nombre del patrón, el mensaje de error de la excepción debería ser del tipo **"El patrón de la embarcación no puede estar vacío, se necesita un patrón para iniciar la navegación."**
- si el número de tripulantes es inferior al mínimo permitido en la clase o superior al máximo permitido en la embarcación, el mensaje de error de la excepción debería ser del tipo **"El número de tripulantes debe estar entre xxx y zzz."**, donde xxx será el mínimo permitido y zzz será el máximo de tripulantes que admite la embarcación.

`parar_navegacion()`, que servirá **para detener la navegación de una embarcación** que se encuentre navegando. Este método recibirá por parámetro el tiempo que ha estado la embarcación navegando y no devolverá nada.

Igual que con el anterior método, tendremos que tener en cuenta si alguno de los parámetros es incorrecto para lanzar las correspondientes excepciones:

- si se intenta parar la navegación de una embarcación que no está navegando, se debería lanzar una excepción con el mensaje **"La embarcación xxx no está navegando."**, donde xxx será el nombre de la embarcación.
- si el tiempo de navegación es negativo, el mensaje de error de la excepción debería ser del tipo **"Tiempo navegando incorrecto, debe ser mayor que cero."**

*5. También implementaremos un **método abstracto para indicar la señalización de las embarcaciones**:*

Para representar los tipos de señalización que tiene cada embarcación implementaremos el siguiente método:

- `señalizar()`, que servirá para indicar el tipo de señalización que tiene la embarcación. Este método no tendrá ningún parámetro y no devolverá nada.

Este método será sobreescrito en cada subclase.

6. Por último, sobrescribiremos el método `__str__` para que devuelva una cadena que represente el estado del barco en un momento dado

El cadena final devuelto por el método debe tener la siguiente estructura:

- la etiqueta **"Nombre de la embarcación: "** junto con el nombre que se le asignó a la embarcación y a continuación una coma (",");
 - la etiqueta **"Tripulación: "** y a continuación el número de tripulantes del barco y a continuación una coma (",");
 - si el barco se encuentra navegando o no indicándolo con el texto **"Sí"** cuando lo esté haciendo y **"No"** cuando no, y a continuación una coma (",");
 - cuando se encuentre navegando, a continuación se mostrará información sobre la navegación que incluirá la etiqueta **"con el patrón"** junto con el **nombre del patrón**, a continuación la etiqueta **"en"** junto con el **rumbo en el que se encuentre navegando** y por último la etiqueta **"a"** junto con la **velocidad en la que se encuentre navegando**, seguida de la etiqueta **"nudos"** para indicar la unidad de medida y todo ello terminando en coma (",");
 - la etiqueta **"Tiempo total de navegación del barco: "** y a continuación el **tiempo (en horas) total de navegación del barco** seguida de la etiqueta **"horas"** para indicar la unidad de medida;
-

1.2.- Interfaces Navegable y Regateable



Interfaces Navegable y Regateable

Se tendrán que implementar las interfaces que se describen a continuación:

1.- Interfaz INavegable

La interfaz **INavegable** será implementada por cualquier embarcación. Para proporcionar esta funcionalidad habrá que escribir los siguientes métodos:

- método `iniciar_navegacion()`, que permite iniciar la navegación.
- método `parar_pavegacion()`, que permitirá detener la navegación.

2.- Interfaz IRegateable

La interfaz **IRegateable** será implementada por cualquier embarcación capaz de participar en una regata (**velero**). Para proporcionar esta funcionalidad habrá que escribir el siguiente método:

- método `iniciar_regata()`, que permite iniciar una regata de un barco con otro.

Todos los detalles de este método los puedes ver en el apartado 1.4.3.

IMPORTANTE: Las características de los métodos de ambas interfaces son las mismas que se indicaron para la tarea de la unidad anterior.

Las clases que incorporen estas interfaces tendrán que implementar estos métodos "a su modo", dependiendo del comportamiento y las características de cada clase específica. Es posible que cada clase tenga que implementarlos de una manera diferente (**polimorfismo**) o no.

1.3.- Clase Lancha



Clase Lancha

Esta clase hereda de la **superclase Embarcación**.

En esta clase definiremos los atributos mínimos para representar las características adicionales que tiene una lancha con respecto a una embarcación básica dentro de nuestro puerto deportivo.

Esta clase **si que debe ser instanciable**, ya que si crearemos objetos de este tipo.

La clase Lancha debe tener sus **atributos, constructores y métodos** que describiremos en los siguientes puntos.



PublicDomainPictures
<<https://pixabay.com/es/photos/puerto-mar%C3%ADtimo-puerto-embarcaciones-3786662/>> (Pixabay License
<<https://pixabay.com/es/service/license/>>)

1.3.1.- Atributos



Atributos

Esta clase debe heredar todos sus atributos de la clase **Embarcacion**, teniendo en cuenta lo siguiente:

*Todos los atributos deben ser **accesibles únicamente por la clase Lancha**, exceptuando las **constantes**, que deben ser **públicas** tal y como se indica más abajo.*

Algunos atributos tendrán un valor que se definirá al instanciarse el objeto y ya nunca modificarán su contenido durante toda la vida del objeto (constantes). Podría decirse que definen algo así como la "naturaleza inmutable" o **características intrínsecas del objeto**:

- **número de motores** de la lancha.
- **cantidad de combustible** en litros.

Además, también se dispondrá de otro atributos que pertenecerá a la clase (atributos de clase), más que a un objeto en particular, compartiendo su valor para todos los objetos:

1. el **número de lanchas** creadas hasta ese momento (equivalente al número de objetos Lancha instanciados).

Por último, junto con esos atributos estáticos variables, la clase también va a contener una serie de **constantes** que serán de utilidad a la hora de realizar comprobaciones o asignaciones por omisión. Estas constantes serán **públicas** y tendrá acceso a ellas cualquier programador que utilice la clase para así poder asegurarse en sus programas de que no se superan los umbrales permitidos:

1. **número mínimo de motores** permitido en las lanchas (1). Atributo MIN_MOTORES;
2. **número máximo de motores** permitido en las lanchas (2). Atributo MAX_MOTORES;
3. **cantidad mínima de combustible** permitido en las lanchas (8). Atributo MIN_COMBUSTIBLE;
4. **cantidad máxima de combustible** permitido en las lanchas (50). Atributo MAX_COMBUSTIBLE;
5. **factor de combustible** consumido por una lancha (0.026). Atributo FACTOR_COMBUSTIBLE;
6. **velocidad mínima** permitida durante la navegación (1 nudo). Atributo MIN_VELOCIDAD_LANCHA;

7. **velocidad máxima** permitida durante la navegación (50 nudos).
Atributo MAX_VELOCIDAD_LANCHA;

Se te proporciona explícitamente el **nombre de estos últimos atributos** porque van a ser **públicos** y ya hay programas desarrollados por otras personas que están usando esos valores (son valores de "interfaz" y, por tanto, públicos). En consecuencia, hemos tenido que ponernos de acuerdo con el resto de programadores del equipo de desarrollo en los nombres de todos aquellos elementos que sean públicos (nombre de la clase, nombre de los atributos públicos, nombres de los métodos públicos, orden y tipo de los parámetros en los métodos públicos, etc.). Para el resto de elementos de la clase (miembros privados, variables locales, etc.) se podrán utilizar los identificadores que consideres más oportunos, pues estarán ocultos a otros programadores y solo serán accesibles desde el código de los métodos de la propia clase.

1.3.2.- Constructores



Constructores

Tendrás que implementar dos **constructores** para la clase Lancha:

1. Un constructor con cuatro parámetros: nombre, número máximo de tripulantes, número de motores, nivel de combustible .

Esta clase deberá llamar al constructor de la clase base **Embarcacion** teniendo en cuenta el **número de motores** y el **nivel de combustible**, por lo que **su constructor debe ser de 4 parámetros**.

La única excepción que debe implementarse en este constructor será:

- si el constructor recibe un número no válido de motores, el mensaje de error de la excepción debería ser del tipo **"El número de motores debe estar entre xxx y zzz."**. Donde xxx y zzz son el número mínimo y máximo de motores.
- si el constructor recibe un número no válido de nivel de combustible, el mensaje de error de la excepción debería ser del tipo **"El nivel de combustible debe estar entre xxx y zzz."**. Donde xxx y zzz son el número mínimo y máximo de combustible.

Si los parámetros son correctos, se procederá a asignar los **valores iniciales de los atributos de objeto** así como a **actualizar los atributos de clase** que sea necesario modificar.

2. Un constructor sin parámetros.

Este constructor generará un objeto Lancha con el **número mínimo de motores**, el **nivel máximo de combustible** y el **número mínimo de tripulantes**, y el **nombre se generará de forma automática**, concatenando la palabra "Lancha" con un espacio y con un código numérico. Este número debe coincidir con el número de lanchas creados hasta el momento, para que no se repita. Ejemplos: "Lancha 1", "Lancha 23", "Lancha 59", etc.

En este caso, no debería existir la posibilidad e lanzamiento de excepciones

1.3.3.- Métodos

Métodos

1. *Añade a la clase Lancha los siguientes **métodos de consulta (getters)** para poder obtener la siguiente información de objeto:*

1. `get_num_motores()`, que devuelva el **número de motores** de la lancha (int);
2. `get_cantidad_combustible()`, que devuelva la **cantidad de combustible** que tiene la lancha (int);

2. *Además de esos métodos consultores sobre el estado del objeto, también habrá que implementar algún **método que devuelva información genérica o global sobre la clase (estáticos)**:*

1. `get_num_lanchas()`, que devuelva la **cantidad de lanchas creadas** hasta el momento (int).

3. *También tendremos que **sobreescribir algunos métodos para modificar atributos de nuestra clase**:*

Para representar el cambio de rumbo de una lancha que está navegando reescribiremos el siguiente método:

- `set_rumbo()`, que servirá para representar el cambio de rumbo de una lancha que está navegando. Este método recibirá por parámetro el nuevo rumbo y no devolverá nada.

En este caso, las posibles excepciones que podrían lanzarse son **las propias que lanza el método definido en la clase base y además:**

- Tendremos que tener en cuenta el parámetro de entrada (ya que si la embarcación está ya navegando o se le intenta poner el mismo rumbo que tenía, ya se controla con este mismo método de la superclase). Debes tener en cuenta que **si el valor del rumbo es nulo**, y habría que lanzar una **excepción** con el mensaje de error "El rumbo no puede ser nulo, debes indicar el rumbo (norte, sur, este u oeste) para poder modificarlo".
- Además, si **el parámetro rumbo es distinto de "norte", "sur", "este" u "oeste"**, también deberá lanzarse una **excepción** con el mensaje de error "El rumbo no es correcto, debes indicar el rumbo (norte, sur, este u oeste) para poder modificarlo".

4. *Además debemos **sobreescribir los métodos de la interfaz "Navegable"**:*

Para poder **iniciar la navegación** y posteriormente **pararla** habrá que sobrescribir los siguientes métodos que se encargarán de actualizar apropiadamente todos los atributos de estado necesarios, tanto de objeto como de clase, para que se realice adecuadamente el proceso de navegación:

iniciar_navegacion(), que servirá para iniciar la navegación de una lancha. Este método recibirá por parámetro la velocidad, el rumbo, el patrón y el número de tripulantes y no devolverá nada.

En este caso, las posibles excepciones que podrían lanzarse son **las propias que lanza el método definido en la clase base** y además tendremos que tener en cuenta si alguno de los **parámetros del método es incorrecto para lanzar las correspondientes excepciones**:

- si se intenta iniciar la navegación de una lancha sin la cantidad mínima de combustible o una cantidad mayor de la permitida, se debería lanzar una excepción con el mensaje **"La lancha xxx no tiene un nivel de combustible válido para iniciar la navegación. El nivel de combustible debe estar entre xxx y zzz."**
- si la velocidad de la lancha no está comprendida entre la máxima y la mínima permitida para las lanchas, y el mensaje de error de la excepción debería ser del tipo **"La velocidad de navegación de xxx nudos asignada a zzz es incorrecta."**, donde xxx es la velocidad de la lancha y zzz es el nombre de la lancha.

parar_navegacion(), que servirá para detener la navegación de una lancha que se encuentre navegando. Este método recibirá por parámetro el tiempo que ha estado la embarcación navegando y no devolverá nada. Adicionalmente a esto, el método debe hacer un cálculo del combustible consumido durante la navegación y actualizarlo en el objeto.

En este caso, las posibles excepciones que podrían lanzarse son **las propias que lanza el método definido en la clase base** y además:

- La **cantidad de combustible tras la navegación** será el resultado que obtenemos de **multiplicar la velocidad de la lancha por el tiempo navegando y por el factor de combustible**. No olvides que la cantidad de combustible es un entero, por lo que tendrás que redondear y hacer algún casting. También ten en cuenta que la cantidad de combustible **no puede ser menor que cero**.

Con este método no tendremos que tener en cuenta ninguna excepción adicional.

5. También tendremos que redefinir el método abstracto definido en la clase Embarcacion para indicar la señalización de las lanchas:

Para representar el tipo de señalización que tienen las lanchas sobrescribiremos el siguiente método:

- `señalizar()`, que servirá para indicar el tipo de señalización que tiene la embarcación. Este método no tendrá ningún parámetro y no devolverá nada pero mostrará por pantalla el mensaje **"AVISO de señalización de la lancha xxx con bocinas y luces intermitentes."**, siendo xxx el nombre de la lancha.

Con este método comprobaremos cómo en tiempo de ejecución se establece la **ligadura dinámica**.

6. Por último, sobrescribiremos el método `__str()` para que devuelva una cadena que represente el estado de una lancha en un momento dado

Este método `__str()` deberá devolver una cadena que represente el **estado de la lancha** en un momento dado. Añadirá a siguiente información a la ya devuelta por la clase Embarcación:

1. la etiqueta **"Número de motores: "** junto al **número de motores de la lancha** y a continuación una coma (",");
2. la etiqueta **"Nivel de combustible: "** junto con el **nivel de combustible actual de la lancha**.

Analicemos un posible ejemplo de salida:

Nombre de la embarcación: Rapidisima, Tripulación: 2, Navegando: Sí, con el patrón Juan Lopez en oeste a 25 nudos, Tiempo total de navegación de la embarcación: 0,42 horas, Número de motores: 1, Nivel de combustible: 30

donde observamos que:

- el **nombre** de la lancha es **Rapidisima**;
- el **número de motores** de la lancha es **1**;
- la **cantidad de combustible** de la lancha es de **30**;
- la **tripulación** de la lancha es **2**;
- la lancha **se encuentra navegando** en este momento;
- la **información sobre la navegación** indica que se está navegando con el patrón **Juan Lopez** en un rumbo de **oeste** y a una velocidad de **25** nudos;
- el **tiempo total de navegación** para el barco es **0,42** horas;

Recuerda que la clase debe llevar un control del número de lanchas que están navegando en cada momento.

*Además, para calcular la cantidad de combustible consumida se aplicará la fórmula: **velocidad*tiempo_navegando*FACTOR_COMBUSTIBLE**.*

1.4.- Clase Velero



Clase Velero

Esta clase también hereda de la **superclase Embarcación**.

En esta clase definiremos los atributos mínimos para representar las características adicionales que tiene un velero con respecto a una embarcación básica dentro de nuestro puerto deportivo.

Esta clase **si que debe ser instanciable**, ya que si crearemos objetos de este tipo.

Además, esta clase debe implementar la interfaz **Regateable**.

La clase Velero debe tener sus **atributos, constructores y métodos** que describiremos en los siguientes puntos.



NoName_13

<https://pixabay.com/es/photos/puerto-mar%C3%ADtimo-puerto-embarcaciones-3786662/>

(Pixabay License

<https://pixabay.com/es/service/license/>)

1.4.1.- Atributos

Atributos

Esta clase debe heredar todos sus atributos de la clase **Embarcacion**, teniendo en cuenta lo siguiente:

Todos los atributos deben ser accesibles únicamente por la clase Velero, exceptuando las constantes, que deben ser públicas tal y como se indica más abajo.

Algunos atributos tendrán un valor que se definirá al instanciarse el objeto y ya nunca modificarán su contenido durante toda la vida del objeto (constantes). Podría decirse que definen algo así como la "naturaleza inmutable" o **características intrínsecas del objeto**:

- **número de mástiles** del velero.

Además, también se dispondrá de otro atributo que pertenecerá a la clase (atributos de clase), más que a un objeto en particular, compartiendo su valor para todos los objetos. Este atributo tendrá sentido siempre, independientemente de que existan o no instancias de la clase (**atributos de clase**) y se accederá a él a través del nombre de la clase (y no de una referencia a un objeto):

1. el **número de veleros** creados hasta ese momento (equivalente al número de objetos Velero instanciados).

Por último, junto con esos atributos estáticos variables, la clase también va a contener una serie de **constantes** que serán de utilidad a la hora de realizar comprobaciones o asignaciones por omisión. Estas constantes serán **públicas** y tendrá acceso a ellas cualquier programador que utilice la clase para así poder asegurarse en sus programas de que no se superan los umbrales permitidos:

1. debe incorporar un **número mínimo de mástiles** permitido en los veleros (1).
Atributo MIN_MASTILES;
2. debe incorporar un **número máximo de mástiles** permitido en los veleros (4).
Atributo MAX_MASTILES;
3. la **velocidad mínima** permitida durante la navegación (2 nudos).
Atributo MIN_VELOCIDAD_VELERO;
4. la **velocidad máxima** permitida durante la navegación (30 nudos).
Atributo MAX_VELOCIDAD_VELERO;

Se te proporciona explícitamente el **nombre de estos últimos atributos** porque van a ser **públicos** y ya hay programas desarrollados por otras personas que están usando

esos valores (son valores de "interfaz" y, por tanto, públicos). En consecuencia, hemos tenido que ponernos de acuerdo con el resto de programadores del equipo de desarrollo en los nombres de todos aquellos elementos que sean públicos (nombre de la clase, nombre de los atributos públicos, nombres de los métodos públicos, orden y tipo de los parámetros en los métodos públicos, etc.). Para el resto de elementos de la clase (miembros privados, variables locales, etc.) se podrán utilizar los identificadores que consideres más oportunos, pues estarán ocultos a otros programadores y solo serán accesibles desde el código de los métodos de la propia clase.

1.4.2.- Constructores



Constructores

Tendrás que implementar dos **constructores** para la clase `Velero`:

1. Un constructor con tres parámetros: nombre, número de mástiles y número máximo de tripulantes.

Este constructor deberá llamar al constructor de la clase base `Embarcacion` teniendo en cuenta el **número de mástiles**, por lo que **su constructor debe ser de 3 parámetros**.

Si alguno de los parámetros del constructor es incorrecto o se produce alguna circunstancia que no permita que se instancie el objeto, deberá lanzarse una excepción y se detendrá el proceso de instanciación del objeto, además, la única excepción que debe implementarse en este constructor será:

- si el constructor recibe un número no válido de mástiles, la excepción será de tipo `IllegalArgumentException` y el mensaje de error de la excepción debería ser del tipo **"El número de mástiles debe estar entre xxx y zzz."**. Donde xxx y zzz son el número mínimo y máximo de mástiles.

Si los parámetros son correctos, se procederá a asignar los **valores iniciales de los atributos de objeto** así como a **actualizar los atributos de clase** que sea necesario modificar.

2. Un constructor sin parámetros.

Este constructor generará un objeto `Velero` con el **número mínimo de mástiles** y el **número mínimo de tripulantes**. El nombre se generará de forma automática, concatenando la palabra "Velero" con un espacio y con un código numérico. Este número debe coincidir con el número de veleros creados hasta el momento, para que no se repita. Ejemplos: "Velero 1", "Velero 23", "Velero 59", etc.

En este caso, no debería existir la posibilidad de lanzamiento de excepciones.

1.4.3.- Métodos



Métodos

1. *Añade a la clase Velero el siguiente **método de consulta (getter)** para poder obtener la siguiente información de objeto:*

1. `get_num_mastiles()`, que devuelva el **número de mástiles** del velero;

2. *Además, también habrá que implementar algunos **métodos que devuelvan información genérica o global sobre la clase (de clase)**:*

1. `get_num_veleros()`, que devuelva la cantidad de veleros creadas hasta el momento.

3. *También tendremos que **sobreescribir algún método para modificar atributos de nuestra clase**:*

Para representar el cambio de rumbo de un velero que está navegando reescribiremos el siguiente método:

- `set_rumbo()`, que servirá para representar el cambio de rumbo de un velero que está navegando. Este método recibirá por parámetro el nuevo rumbo y no devolverá nada.

En este caso, las posibles excepciones que podrían lanzarse son **las propias que lanza el método definido en la clase base** y además:

- Tendremos que tener en cuenta el parámetro de entrada (ya que si la embarcación está ya navegando o se le intenta poner el mismo rumbo que tenía, ya se controla con este mismo método de la superclase). Debes tener en cuenta que **si el valor del rumbo es nulo**, y habría que lanzar una **excepción** con el mensaje de error "El rumbo no puede ser nulo, debes indicar el rumbo (ceñida o empopada) para poder modificarlo".
- Además, si **el parámetro rumbo es distinto de "ceñida" o "empopada"**, también deberá **lanzarse una excepción** con el mensaje de error "El rumbo no es correcto, debes indicar el rumbo (ceñida o empopada) para poder modificarlo".

4. *Además, debemos **sobrescribir algunos métodos de acción** de las interfaces "Navegable" y "Regateable":*

`iniciar_navegacion()`, que permite iniciar la navegación.

El método `iniciar_navegacion` recibirá como parámetros la velocidad, el rumbo, el nombre del patrón y el número de tripulantes, y no devolverá nada. Si alguno de

los **parámetros del método** `iniciar_navegacion` **es incorrecto**, deberá **lanzarse una excepción**. En este caso, las posibles excepciones que podrían lanzarse son **las propias que lanza el método definido en la clase base y además**:

- si la velocidad de navegación no está entre los valores permitidos en la clase, el mensaje de error de la excepción debería ser del tipo **"La velocidad de navegación de xxx nudos es incorrecta."**, donde xxx será la velocidad introducida.

`iniciar_regata()`, que permite iniciar la regata.

Para representar la competición entre dos veleros habrá que implementar el método `iniciar_regata`, que recibirá como parámetro **un objeto de tipo `velero`**, que será el barco contra el que quiere competir el velero actual.

- Si no se le pasa parámetro, deberá devolver una excepción con el siguiente mensaje de error **"El barco con el que se intenta regatear no existe"**.
- Tenemos que tener en cuenta es que no se podrá iniciar una regata si el velero actual no está navegando. Tampoco se podrá iniciar una regata si el otro velero no está navegando. En caso de que se intentara, debería lanzarse una excepción con el siguiente mensaje de error **"No se puede iniciar la regata, el barco xxx no está navegando."**, donde xxx será el nombre del barco.
- Tampoco podrá realizarse la regata si ambos veleros llevan rumbos distintos en la navegación. En caso de que se intentara, debería lanzarse con el siguiente mensaje de error **"No se puede iniciar la regata, los barcos xxx y zzz deben navegar con el mismo rumbo."**, donde xxx y zzz será el nombre de los veleros.
- De igual modo, si los veleros no tienen el mismo número de mástiles tampoco podrán competir. En caso de que se intentara, debería lanzarse una excepción con el siguiente mensaje de error **"No se puede iniciar la regata, los barcos xxx y zzz no tienen el mismo numero de mástiles."**, donde xxx y zzz será el nombre de los veleros.

Si se puede realizar la regata, el barco cuya velocidad de navegación sea mayor será el ganador de la regata, y se deberá mostrar el mensaje **"El barco xxx ha llegado antes a la línea de llegada."**, donde xxx será el nombre del barco. Si ambos barcos navegan con la misma velocidad empatarán, y se deberá mostrar el mensaje **"Los barcos xxx y zzz han llegado a la vez a la línea de llegada."**, donde xxx y zzz serán los nombres de los barcos.

Recuerda no imprimir por pantalla directamente en los métodos, para ello construye una cadena con el resultado y devuélvela.

5. También tendremos que redefinir el método abstracto definido en la clase `Embarcacion` para indicar la señalización de los veleros:

Para representar el tipo de señalización que tienen los veleros sobreescibiremos el siguiente método:

- `señalizar()`, que servirá para indicar el tipo de señalización que tiene la embarcación. Este método no tendrá ningún parámetro y no devolverá nada pero mostrará por pantalla el mensaje **"AVISO del velero xxx con banderas de señalización marítima."**, siendo xxx el nombre del velero.

Con este método comprobaremos cómo en tiempo de ejecución se establece la **ligadura dinámica**.

6. Por último, existirá un método para **`__str__`**:

Este método **`__str__`** deberá devolver una cadena que represente el **estado del velero** en un momento dado. Añadirá a siguiente información a la ya devuelta por la clase Embarcación:

1. la etiqueta ***"Número de mástiles: "*** junto al **número de mástiles del velero** y a continuación una coma (",");

Atlantis, Tripulación: 1, Navegando: Sí, con el patrón Pepe Martinez en empopada a 10 nudos, Tiempo total de navegación de la embarcación: 1,00 horas, Número de mástiles: 1

donde observamos que:

- el **nombre** del barco es **Atlantis**;
- la **tripulación** del barco es **1**;
- el barco **se encuentra navegando** en este momento;
- la **información sobre la navegación** indicará que se está navegando con el patrón **Pepe Martínez** en un rumbo de **empopada** y a una velocidad de **10 nudos**;
- el **tiempo total de navegación** para el barco es **1 hora**;
- el **número de mástiles** del barco es **1**;

*Recuerda que la clase debe llevar un control del **número de veleros que están navegando** en cada momento.*

1.5.- Programa Principal

En los siguientes subapartados tienes ejemplos de solución de todos los ejercicios propuestos.



Programa Principal

El programa principal deberá hacer las siguientes comprobaciones:

1. Comprueba los constructores y atributos para los veleros y lanchas.
 2. Comprueba los distintos métodos *getters* y métodos de clase
 3. Comprueba el inicio y parada de la navegación de los veleros y de las lanchas.
 4. Comprueba el establecimiento de distintos rumbos a veleros y lanchas.
 5. Comprueba las regatas y la señalización.
 6. Comprueba los métodos `__str__`.
-